

Trends in (big) data Management and Integration: Data Lakes and similar architectures

The Need for Modern (Big) Data Architectures

- Limitations of traditional architectures (bottlenecks, rigid ETL, high cost)
- Explosive growth of heterogeneous data (SaaS, IoT, logs, events, AI-generated)
- New high-complexity use cases (real-time analytics, ML feature stores, generative AI)
- Organizational misalignment between domain teams and central data teams
- Cloud-native paradigm: separation of storage/compute, elastic engines

Data Integration

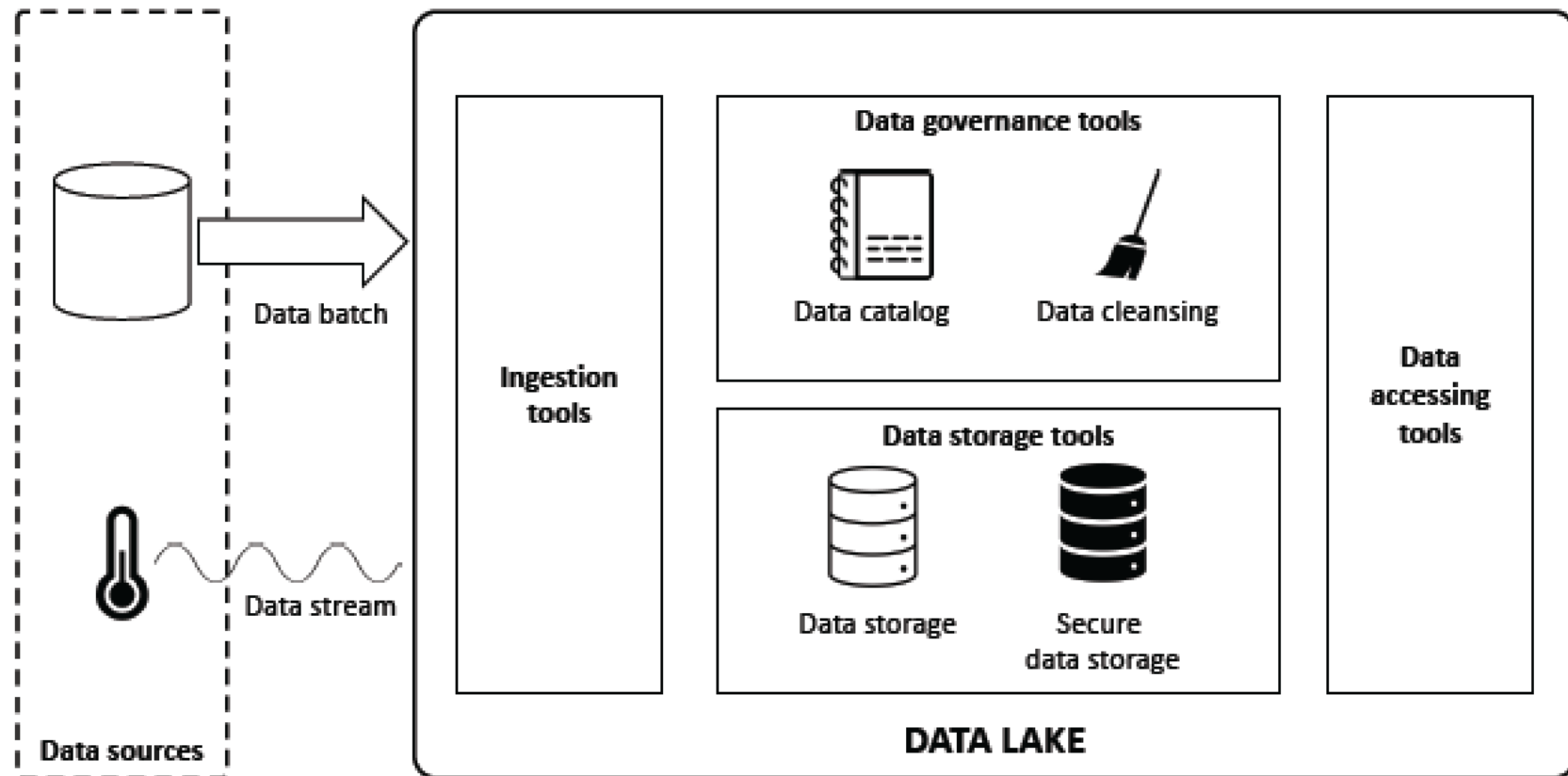
- A fascinating and ever-challenging problem, lending itself to a multitude of solutions
- A still challenging problem, in terms of infrastructure, integration, data cataloguing, data movement and privacy.
- Big Data Integration: still a BIG problem

The Data Lake

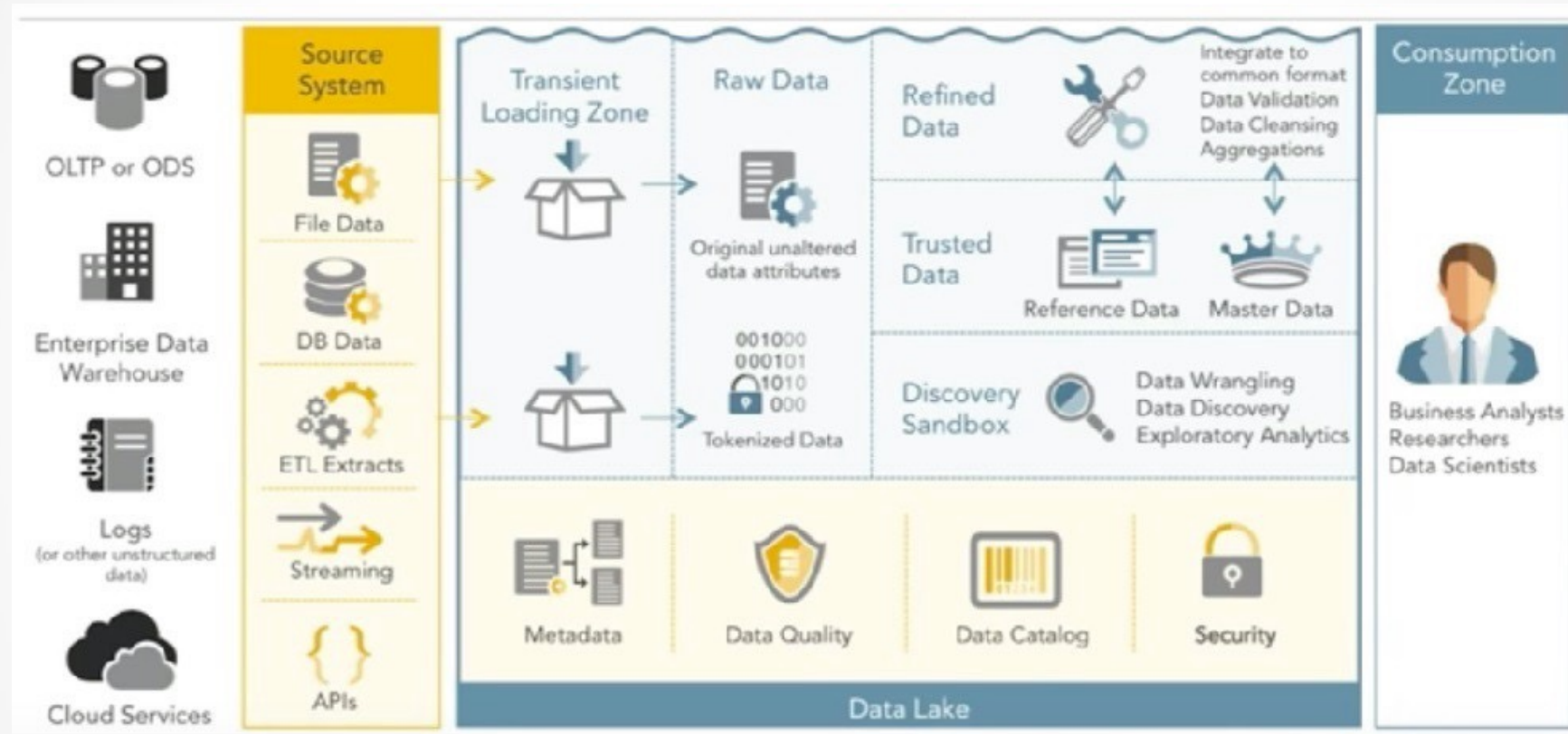
For integrating, many organizations rely on a data warehouse, which is inadequate since it lacks the necessary flexibility because of:

- Its inherently centralized architecture
- The variability of the required data analysis
- The needs related to increasing volume and variety.

Data Lake Minimal Architecture

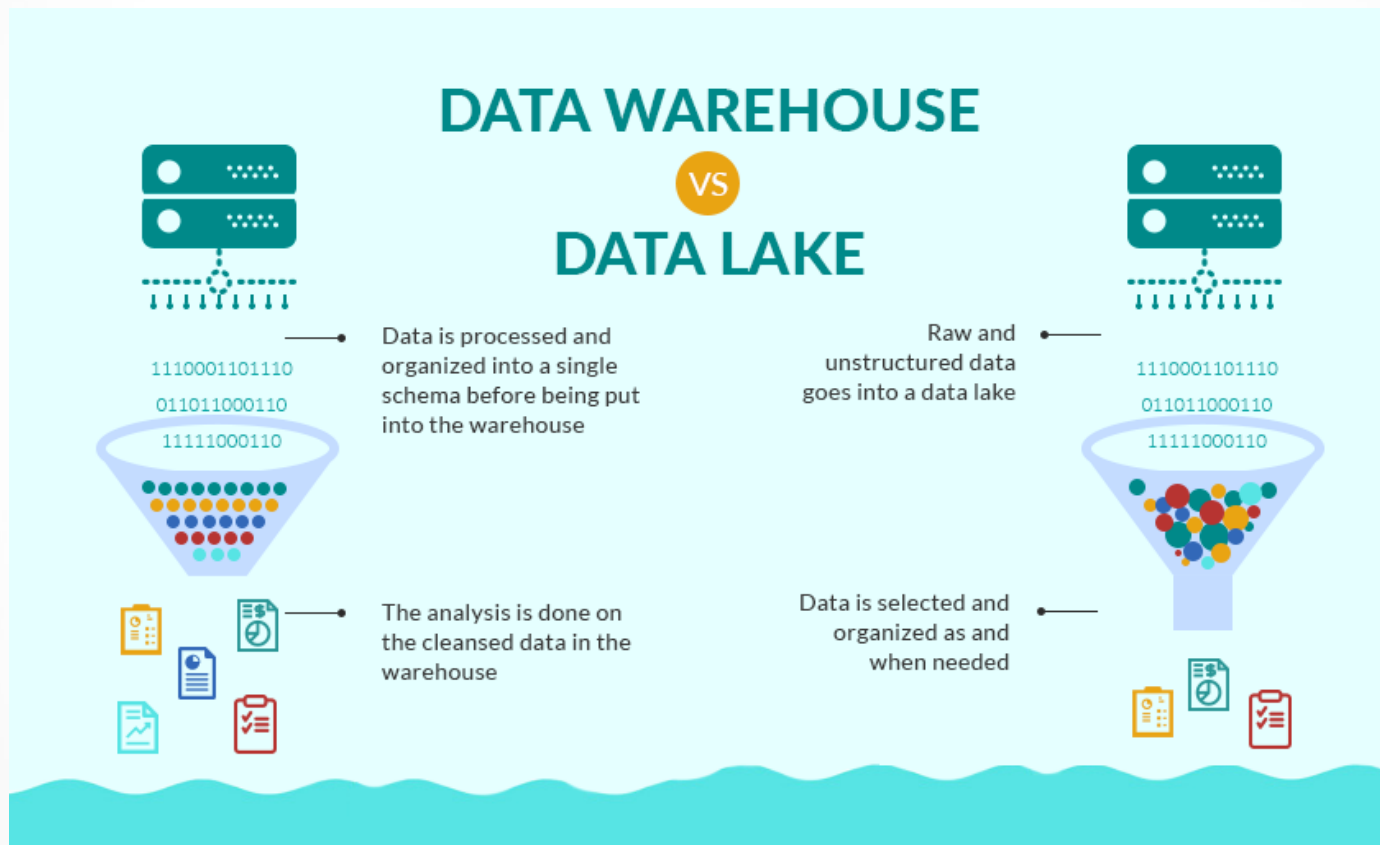


Data lake reference architecture

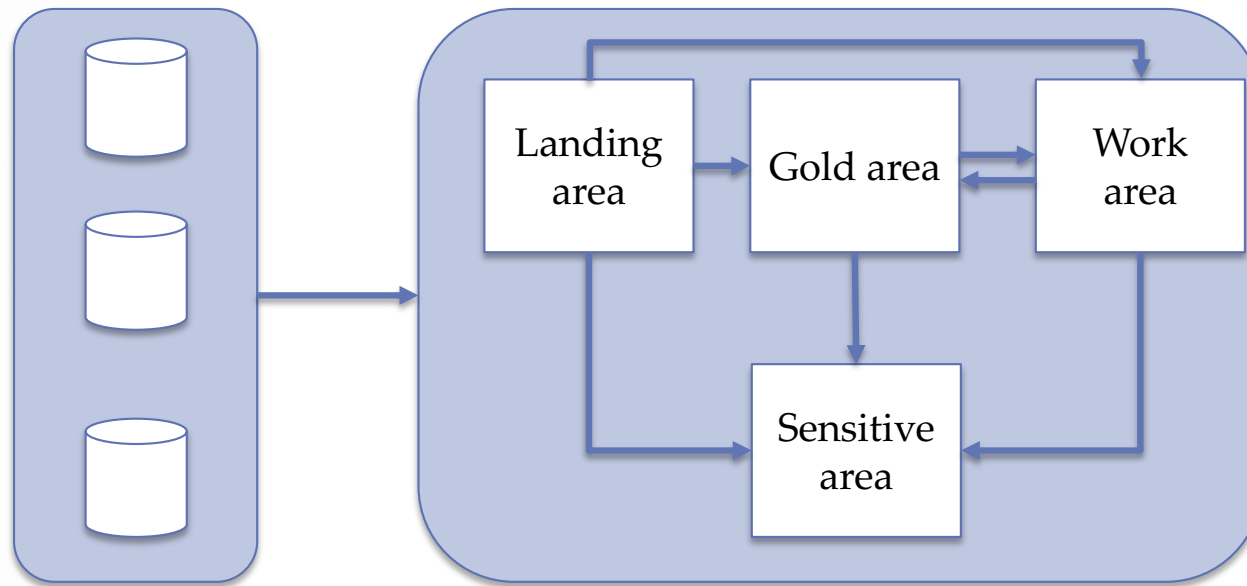


<https://medium.com/@rathi.ankit/data-lakes-in-modern-data-architecture-90d8c38010d>

Data Lake



Organizing the Data Lake



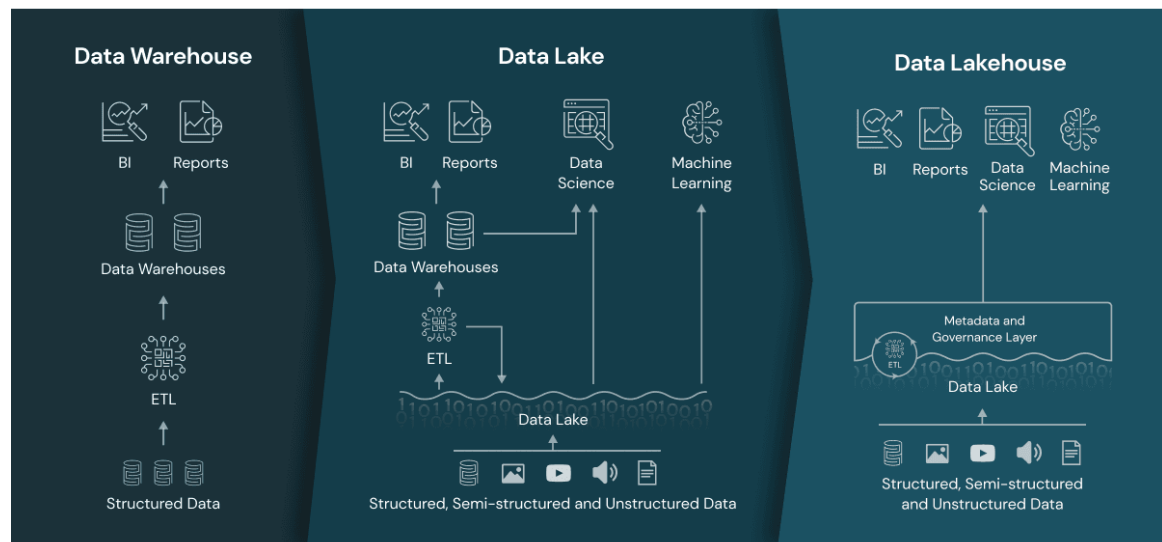
Alex Gorelik, The Enterprise Big Data Lake,
O'reilly 2019

Data Lake design

- Data Lakes can be designed on premises or in cloud
- A Data Lake can be realized also as a virtual data structure (ingest on demand)

Data lakehouse

- A data lakehouse is a data platform that combines the flexible data storage of data lakes with the high-performance analytics capabilities of data warehouses
- Data governance layer is fundamental
 - Data cataloguing
 - Data curation
 - Metadata definition
 - Data access policies



Source: <https://www.databricks.com/it/glossary/data-lakehouse>

Metadata

- A data lake can easily become a data swamp
- A data lake is not simply a storage, requires good practices to register them
- Metadata are **keystones in this practice**

Metadata Type	Main Focus
Technical	Structure, format, storage
Business	Meaning, definitions, ownership
Operational	Usage, performance, refresh
Lineage/Provenance	Data flow, audit trail, dependencies
Security	Access, classification, privacy
Discovery (optional)	Search, tagging, popularity
AI/ML (optional)	Model lifecycle, features, accuracy

Types of Metadata

- **Descriptive metadata** add information about who created a resource, what it is about and what it includes. This is best applied using semantic annotations.
- **Structural metadata** include data about the way data elements are organized, their relationships and the (possible) structure they exist in.
- **Administrative metadata** provide information about the origin of resources, their type and access rights.
- **Semantic metadata** to interpret the meaning of the data via references to concepts, often formally described in a knowledge graph or ontology.

A different, more coarse-grained classification considers:

- **Technical Metadata**, related to the profile of the datasets (e.g., list of attributes, data types, statical information), and
- **Business Metadata**, that provide a domain-aware description of a dataset.

Technical metadata

- **Data Profiling:** the activity that extracts metadata from data analysis. Data profiling highlights the schema, data types, formats used, maximum, minimum, average (if numeric) values, and other information such as:
 - Cardinality (number of unique values per attribute)
 - Selectivity (a measure of the uniqueness of values, measured by dividing cardinality by the number of rows)
 - Density (number of nulls per attribute)

Business Metadata

- Business metadata are often derived from glossaries, taxonomies and ontologies
- Business ontologies are often very complex. That is why in their place many times so-called folksonomies are used: user-built ontologies in which recurring terms are usually used and classified.

Tagging

- Tagging is the process by which metadata is associated with data sources.
- Tagging can be:
 - Manual
 - Crowdsourced based
 - Automatic

Other relevant metadata

- Metadata related to security and data protection
- Data Quality
- Data Lineage (or Provenance)

Data cataloguing

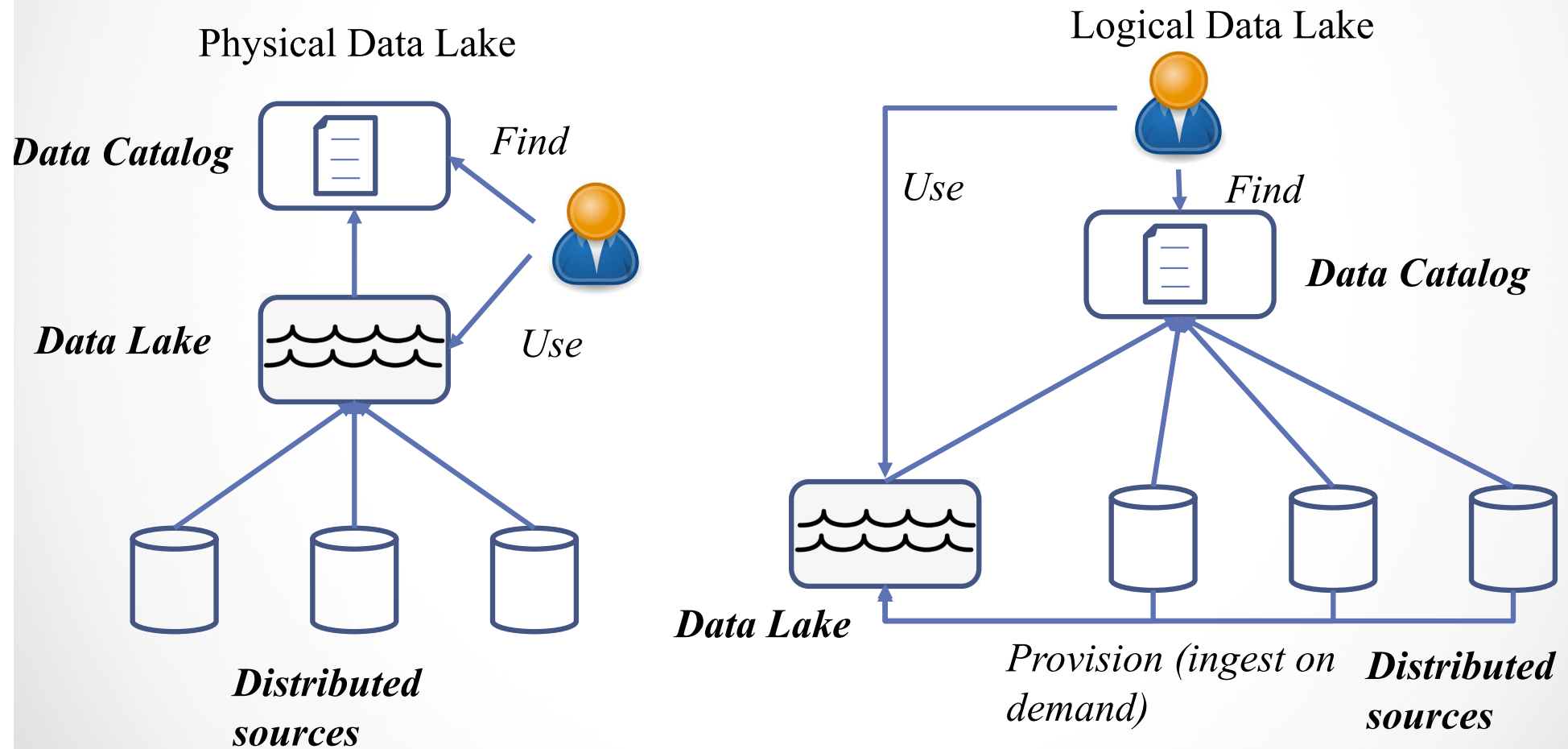
A central tool in data lakes:

- Contains information about the stored data sets
- Enable the discoverability
- It is driven by the metadata associated to the data sets

- A data lake lacks a native catalogue and needs an external one.
- The data lakehouse almost always integrates a data catalog as a native component.

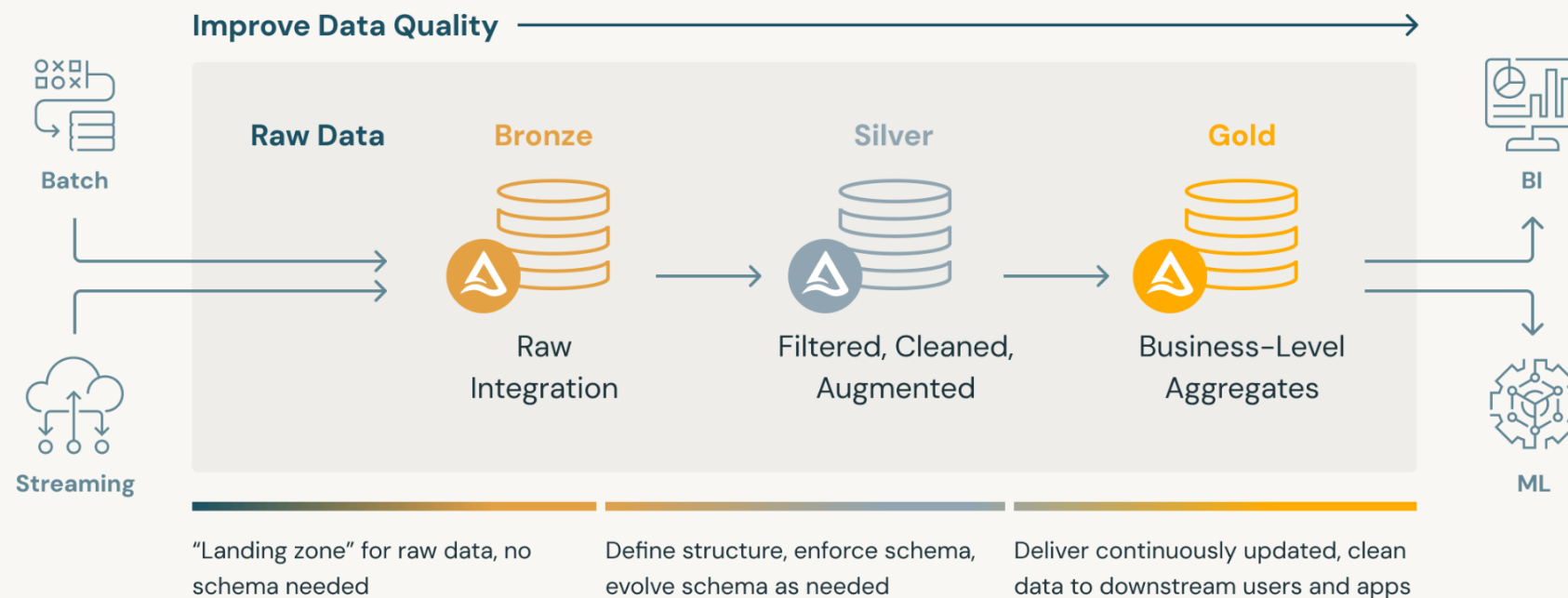
Functionality	Description
Data discovery	Find and explore data assets across the organization
Metadata management	Centralized storage of all types of metadata
Business glossary	Standardized business definitions linked to data
Data lineage	Visual mapping of data flow and dependencies
Data profiling	Automated stats for quality assessment
Governance & security	Classification, policies, ownership, access control
Collaboration	User comments, tags, ratings, annotations
Integrations & APIs	Connect to data sources and tools for automation

Role of the Data Catalog



An example for improving quality: medallion architecture

Building reliable, performant data pipelines with  **DELTA LAKE**

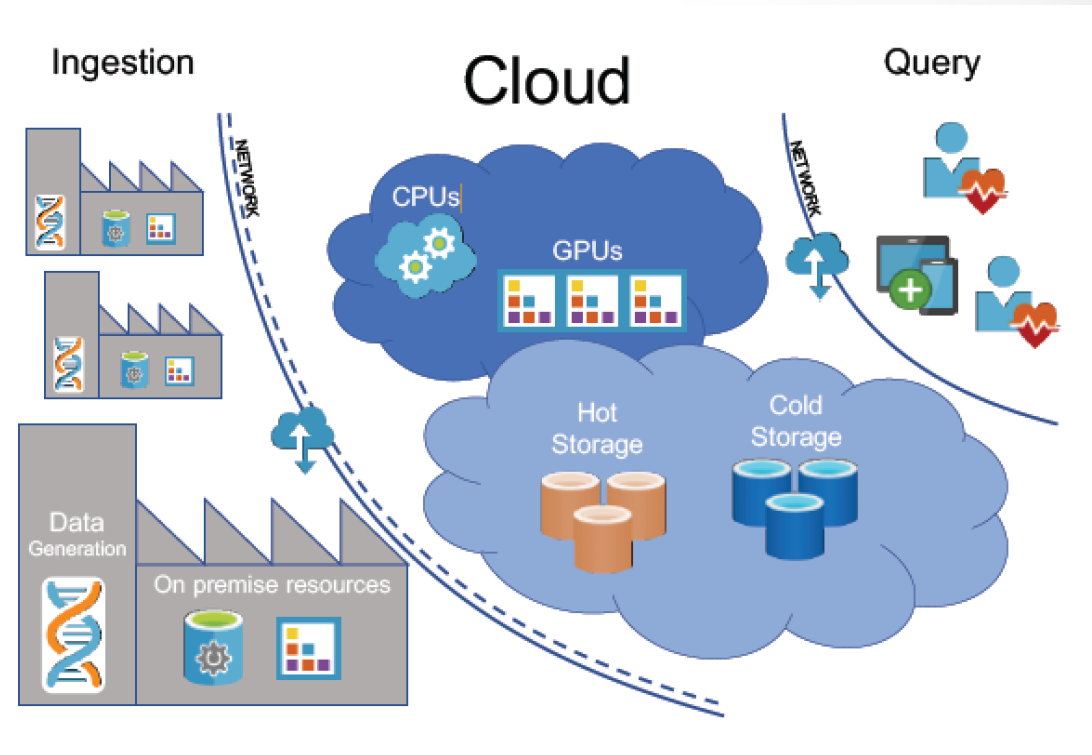


Medallion architecture layers

- BRONZE LAYER (raw data)
 - replicate the raw structure of the raw data with some additional metadata
- SILVER LAYER (cleansed and conformed data)
 - data undergoes a series of operations to a “just-enough” state (cleansing, verification, conforming, integration)
- GOLD LAYER (curated business-level tables)
 - data is restructured into subject area specific databases, designed for specific purposes
 - It contains highly curated and quality data ready for consumption

Infrastructure

- Depending on the types of data at stake, computational resources may require **CPU or GPU support**
- Storage should be organized into a **Hot Storage**, containing data requiring quick access, and a cheaper and slower **Cold Storage** for less frequently used data
- Data analysis can be performed both **on-premises** and **on the cloud**;
- The network is used for **ingestion** and for **querying**: the former requires resources several orders of magnitude larger



Comparison

	Data warehouses	Data lakes	Data lakehouses
Types of data	Mostly structured data	Raw structured, unstructured and semistructured data	Raw or processed structured, unstructured and semistructured data
Storage type	Relational databases	Cloud object storage	Cloud object storage
Storage cost	High	Low	Low
Native query performance	High	Low	High
Schema approach	Schema-on-write	Schema-on-read	Supports both schema-on-write and schema-on-read
Defining feature	Built-in analytics engine supports high performance	Low-cost, scalable storage for all data types	Warehouse performance and lake storage in one solution
Common use cases	Business intelligence and data analytics	Artificial intelligence, machine learning and data science	Business intelligence, data analytics, artificial intelligence, machine learning and data science

Source: <https://www.ibm.com/it-it/think/topics/data-warehouse-vs-data-lake-vs-data-lakehouse>

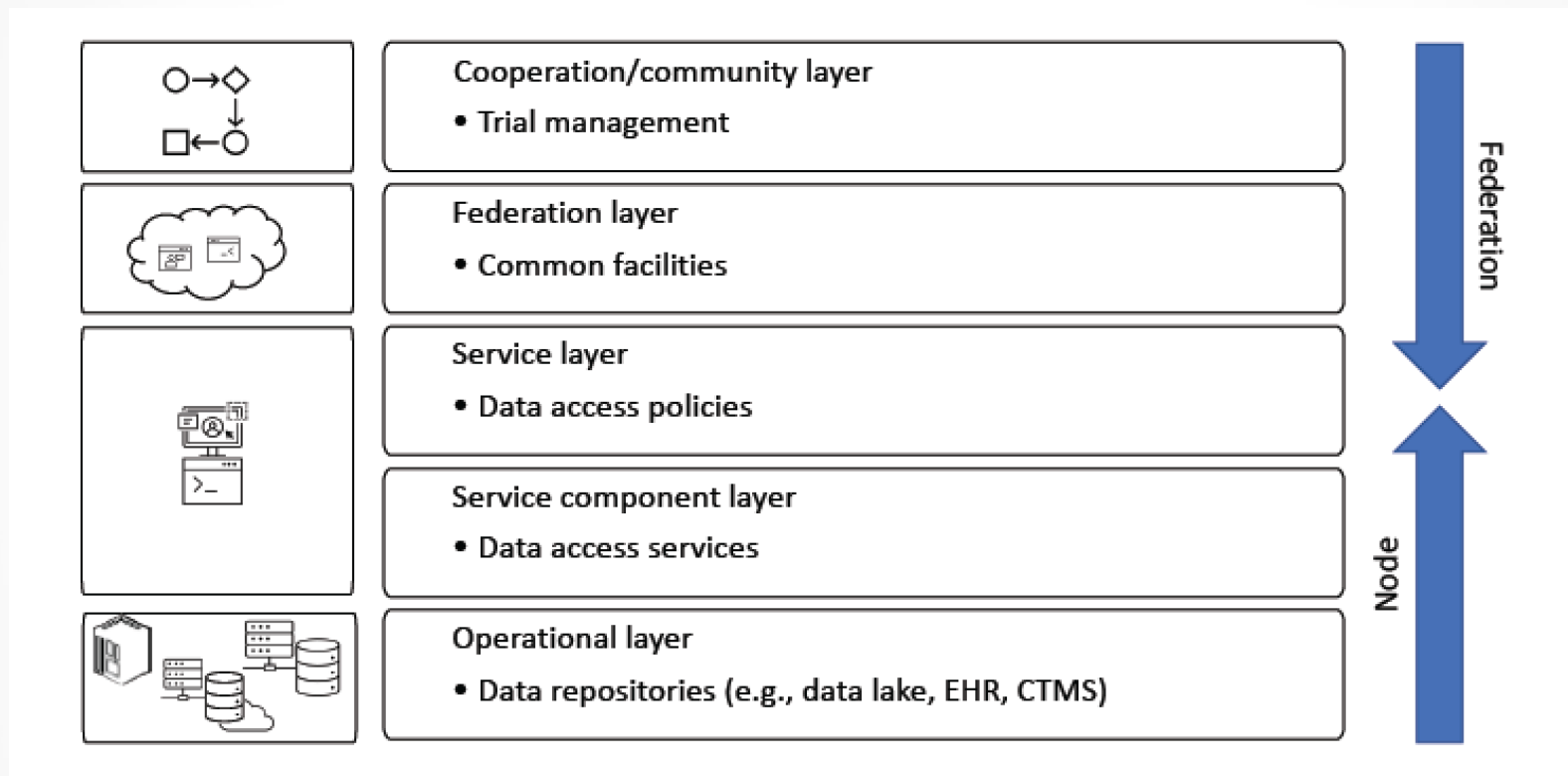
Data lake federation

Adoption of a data lake federation through which the organizations could achieve further benefits by sharing data.

The main research challenge for guaranteeing data reliability is accurate data description, to:

- document their quality,
- facilitate data discovery,
- define security and privacy policies.

Data Lake Federation



Data Lake Federation: Node level

- **Operational layer:** organizations, companies, associates,have implemented their own data repositories to store their data. These might be deployed on the organization's premises or on cloud resources, possibly already organized in a data lake or other data organization, or using a hybrid solution.
- **Service component layer:** it supports the implementation of the data-access services, in charge to expose the selected datasets. For each service, depending on the nature of the data and the type of support that the node wants to offer, e.g., a simple FTP connection, a REST API.
- **Service layer (middle layer):** it implements the policies that regulate the access to the exposed datasets through the defined services. These policies specify who has the right to access what, and also which type of analysis can be performed on a given dataset or the transformations to be applied (e.g., anonymization) before making it available to the requester.

Data Lake Federation: Federation level

- **Federation layer:** it offers both infrastructural and application services: assuming that a set of nodes offers data services according to the structure just described, the data lake federation has the role to create an ecosystem that enables data sharing. For instance, a community cloud can be established to offer common storage and computation facilities that can be used by a node to (temporarily) store data in a convenient place to increase the performance. At the same time, a distributed monitoring system can be offered, to check if the occurring data-sharing is respecting the defined access policies.
- **Cooperation/community layer:** it offers the tools to define and manage multi-centric operations, i.e., where the analysed data could be stored in more than one node.

Pay-as-you-go approach in the data lake(s)

When a query or an update is submitted to an integration system, this has to perform some **reformulation** in terms of a set of queries over the single datasets:

- The system determines **which datasets are needed** to answer the query
- If more than one dataset is needed, determine **which predicates (conditions)** apply to **only a single dataset** and which predicates apply to **elements from more than one dataset**.
- The latter can only be evaluated over all the involved datasets, appropriately combined, whereas each of the former can be evaluated over the specific dataset addressed by that (sub) query.

In a data lake, these operations are performed in **a pay-as-you-go fashion**, avoiding the creation of mediated schemas or the permanent establishment of relationships between the elements of different sources (entity resolution and data fusion), and relying instead on temporary links based on metadata.

Basic tools:

- **keyword search** over a collection of data coupled with effective data visualization
- **improving the metadata** in the system to the end of supporting and validating schema and instance, instance and reference reconciliation

Integrating heterogeneous data in Data Lakes

- How does the Data Lake 'put together' all the data related to a specific query?
- How do we compare data of any kind (e.g., text, images, time series,...) to the end of deriving their similarities and differences?
 - The integration of natural language text can be supported by semantic techniques (ontologies are numerous and rich in the health domain) and/or on ML tools (e.g. word and sentence embeddings)
 - Queries that require to retrieve images, or time series, that show some resemblance, might be based on keyword-search on the image features or other metadata, or exploit some service based on image or plot similarity
 - The data lake should be able to establish a relationship between datasets that have different characteristics, and also allow different modes of interaction
 - Fundamental a catalog for describing data and services
 - This information is the basis to allow search, query and update over all the contents of the data lake, regardless of their formats

DWs vs Data Lakes (aka “Spaces”)

Data Warehouse	Dataspace
Usually Relational DBMS	Often HDFS/Hadoop
Data of recognized high value	Candidate data of potential value
Aggregated data	Detailed source data
Data conforms to enterprise standards	Fidelity to original format and condition (transformed/cleaned later, on demand, when needed)
Known entities	Raw material (metadata) to discover entities
Data integration up front	Data integration on demand
Schema on write	Schema on read (when doing analytics)

The Data Mesh

“Decentralized, Socio-Technical approach to share, access, and manage analytical data in complex and large environments – within or across organizations”

Z. Dehghani, Data Mesh. Delivering Data-Driven Value at Scale, O'Reilly, 2022

Data Mesh Principles(1)

- Domain ownership

- ownership shifts from the centralized ETL pipelines to decentralized units closer to the data following the principles of Domain-Driven Design

- Data as a product

- data teams must apply “product thinking” to the provided datasets
- ensuring discoverability and quality-control.
- DATSIS principles: Discoverability, Addressability, Trustworthiness, Self-describing, Interoperability and Security

Data Mesh Principles(2)

- Self-serve data platform

- The data product lifecycle should be supported by a self-serve data platform, providing the needed tools and infrastructures and accessible without specialized knowledge

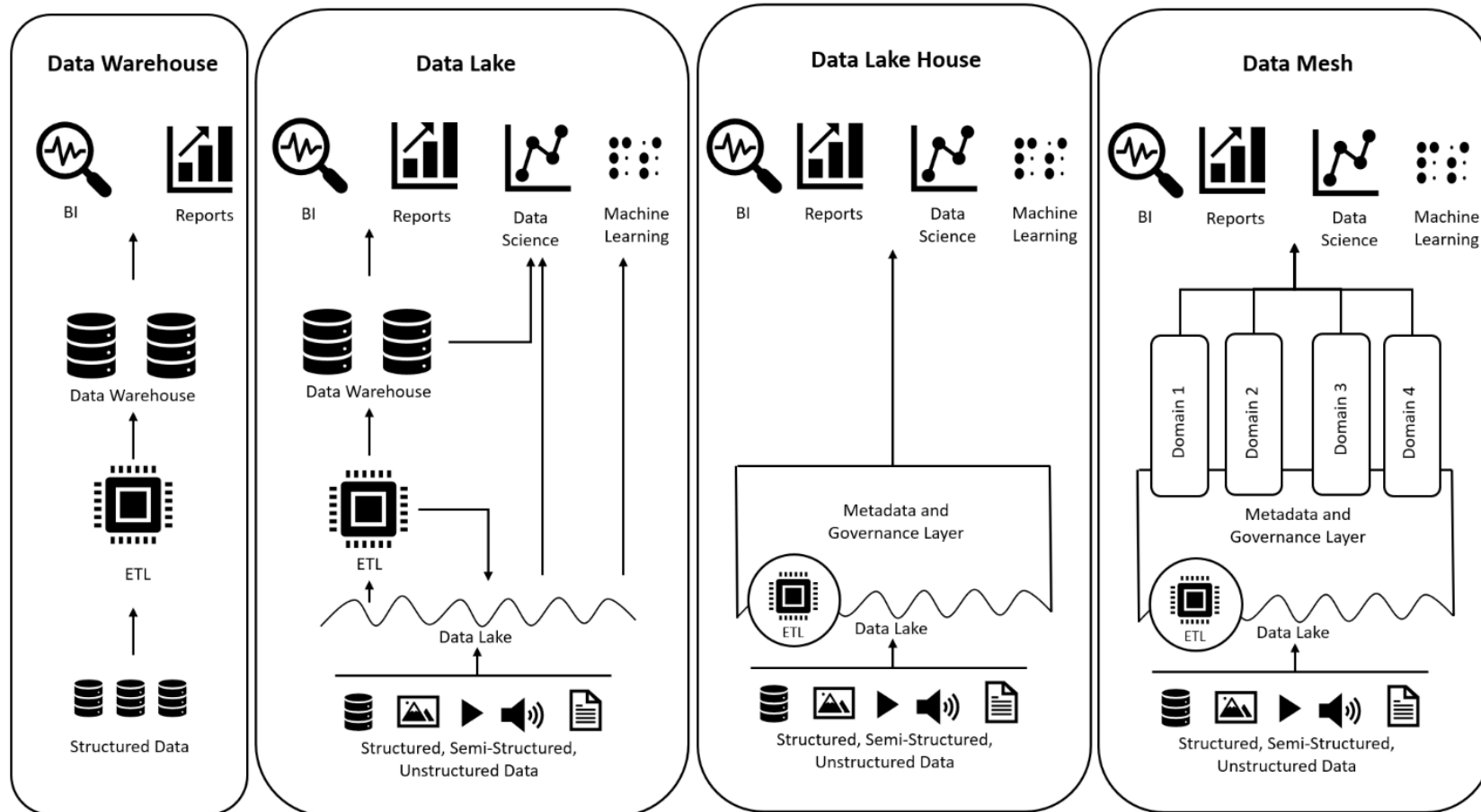
- Federated Computational Governance

- Ensuring data ownership, standardization, interoperability and automated decision execution

Domain ownership

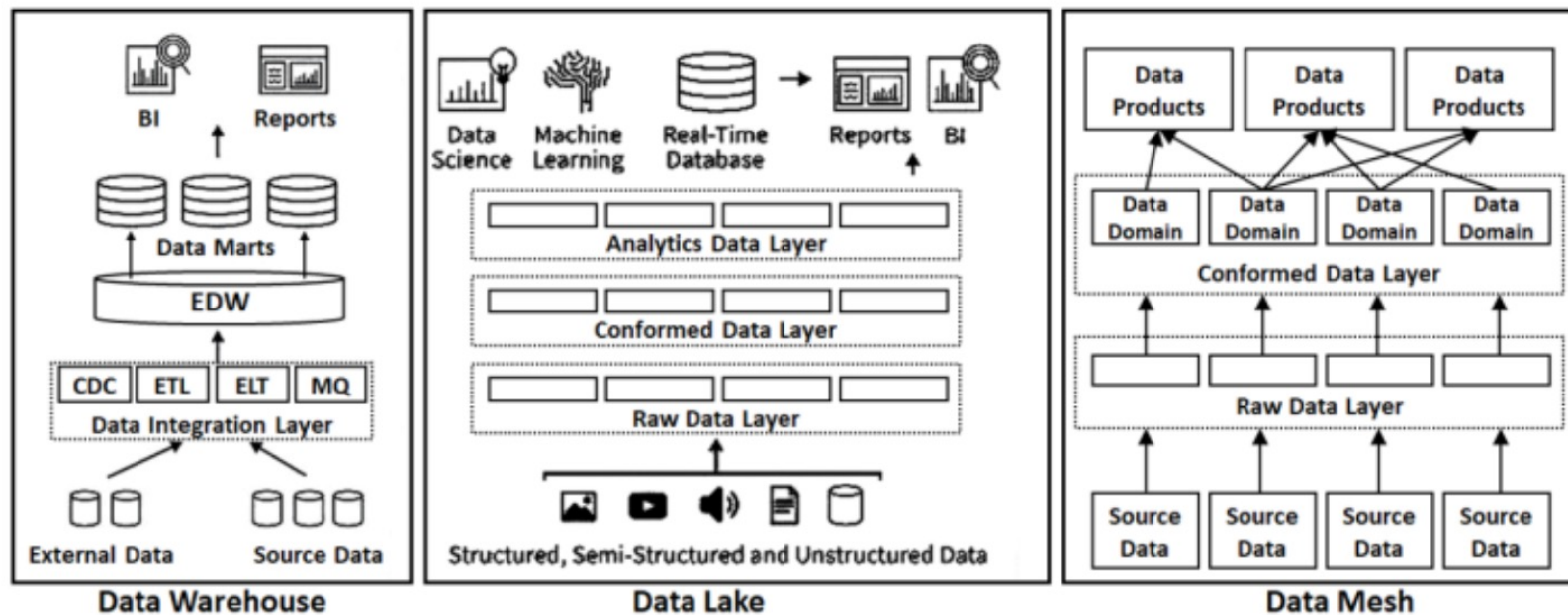
- Domain ownership in a Data Mesh means that each business domain is responsible for producing and maintaining its own high-quality, usable data products, following shared governance standards
- Each business domain (e.g., Sales, Marketing, Finance, Supply Chain) is responsible for:
- **1. Owning its data end-to-end:** A domain owns the full lifecycle of its data (e.g., Data creation, Data quality, Data documentation, Data pipelines, Storage and serving, Access controls)
- **2. Producing “data products”:** Domains don't just produce raw tables. They must provide **high-quality, discoverable data products** designed for others to use.
- **3. Cross-functional domain teams:** A domain team includes: data engineers, analysts / domain experts, data product owner
- **4. Interoperability and standards:** Even though domains are autonomous, they must follow **federated governance** rules: global naming standards, security & privacy standards, interoperability formats, lineage requirements, metadata requirements. This ensures that all domain-owned data can still work together across the company.

Data Mesh



<https://www.linkedin.com/pulse/data-warehouse-vs-lake-lakehouse-mesh-ashish-bijawat/>

Another comparison (1)



<https://www.linkedin.com/pulse/data-warehouse-mesh-making-right-choice-chandra-kallur/>

Data must be FAIR

The FAIR principles are designed to maximize the use and reuse of research data, both by humans and machines, by improving discoverability, allowing integration, and promoting wider sharing, even when adhering to ethical, legal, or contractual restrictions.

- **Findable:**

Data should have a unique and persistent identifier (like a DOI) and be discoverable through rich metadata, making it easy for others to locate.

- **Accessible:**

Data should be retrievable through standardized communication protocols. A persistent identifier and the ability to access the data (even if permission is required) are key.

- **Interoperable:**

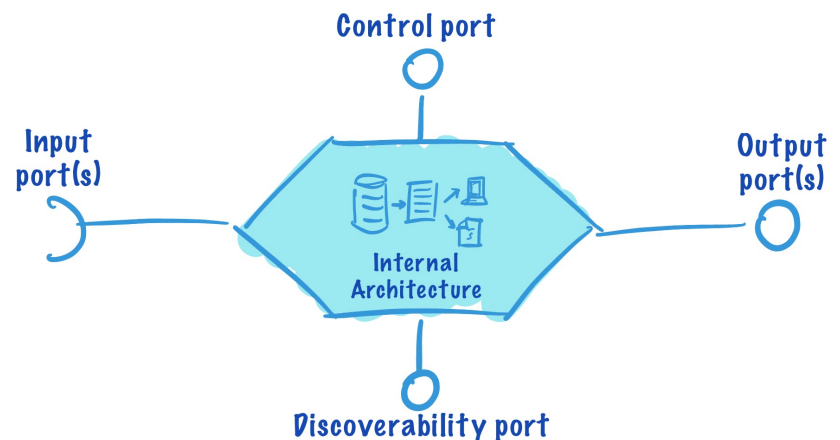
Data should use a common vocabulary and be in a format that allows it to be easily integrated or exchanged with other data. This is especially important for machine processing.

- **Reusable:**

Data should be clearly licensed to specify reuse terms, and it should have the necessary documentation to enable its reuse for new research.

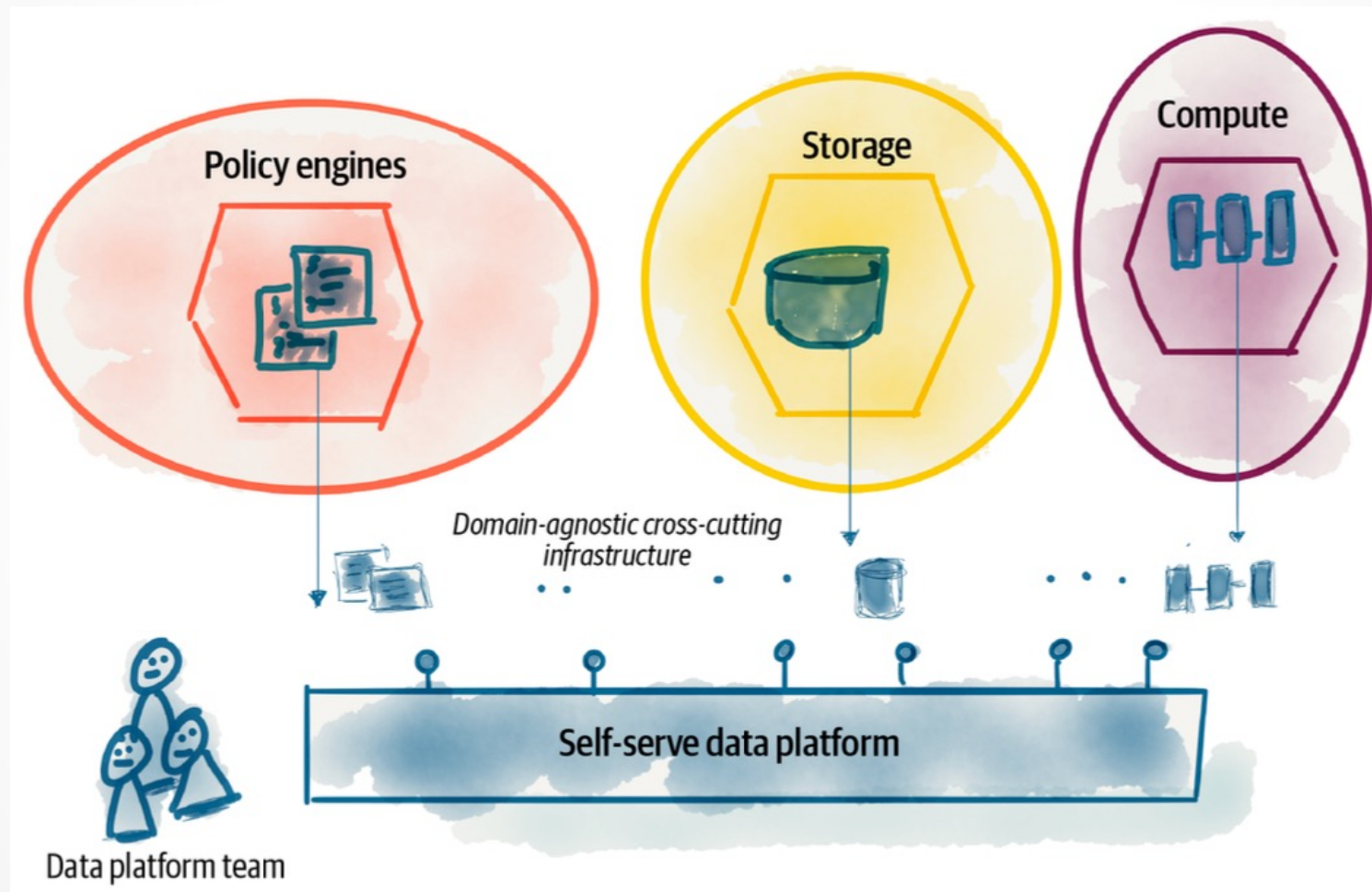
Data as product

- The Data Product is the architectural quantum (minimal unit)
- A data product typically includes:
 - Clean, modeled data
 - Clear SLAs (Service Level Agreements)
 - Metadata + documentation
 - APIs or queries to access it
 - Quality tests
 - Ownership contacts



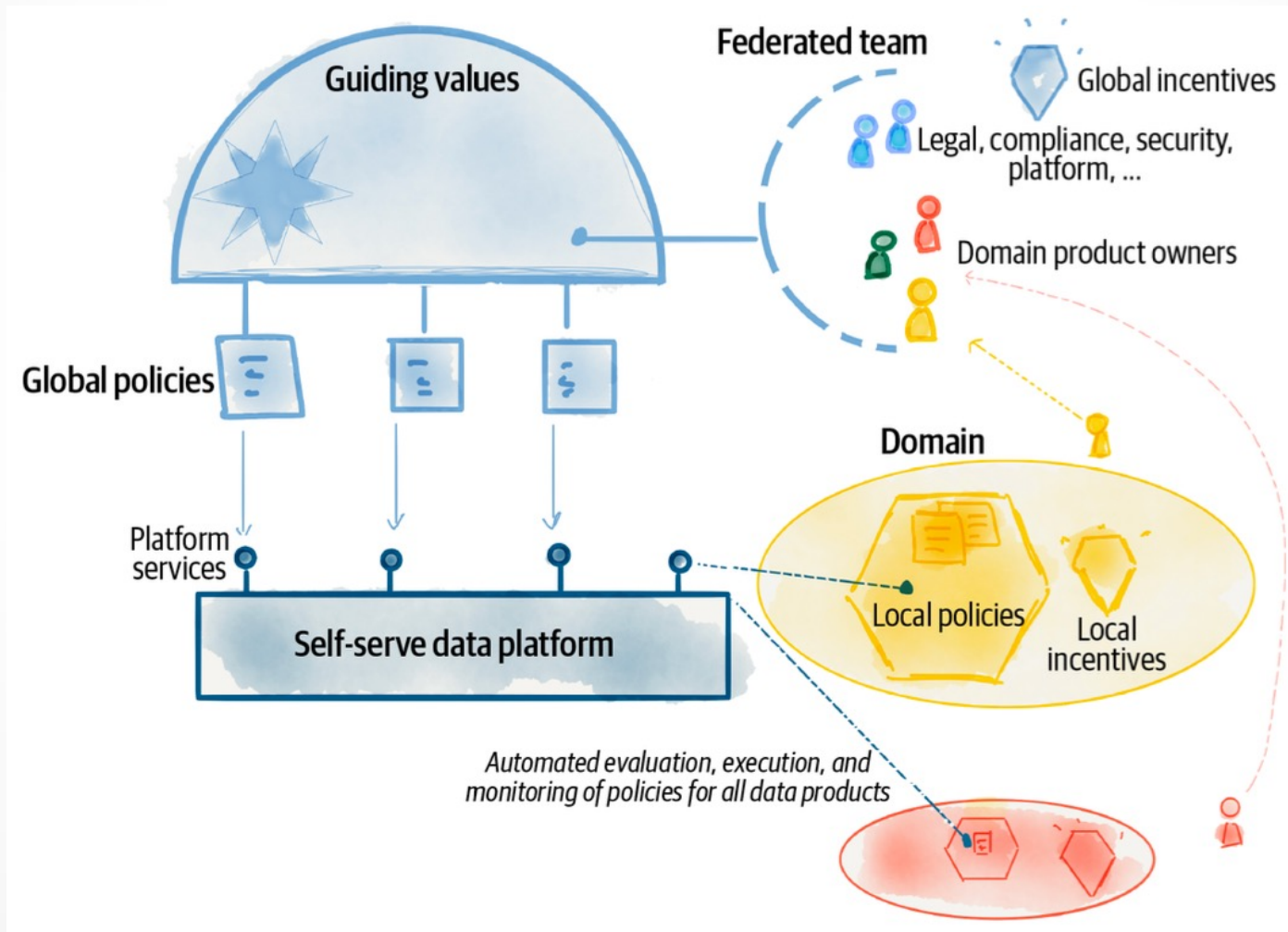
Source: G. Lombardo, Data Friction in Data Sharing: a Physics Inspired Model, Thesis, Politecnico di Milano, 2023

Self-serve platform



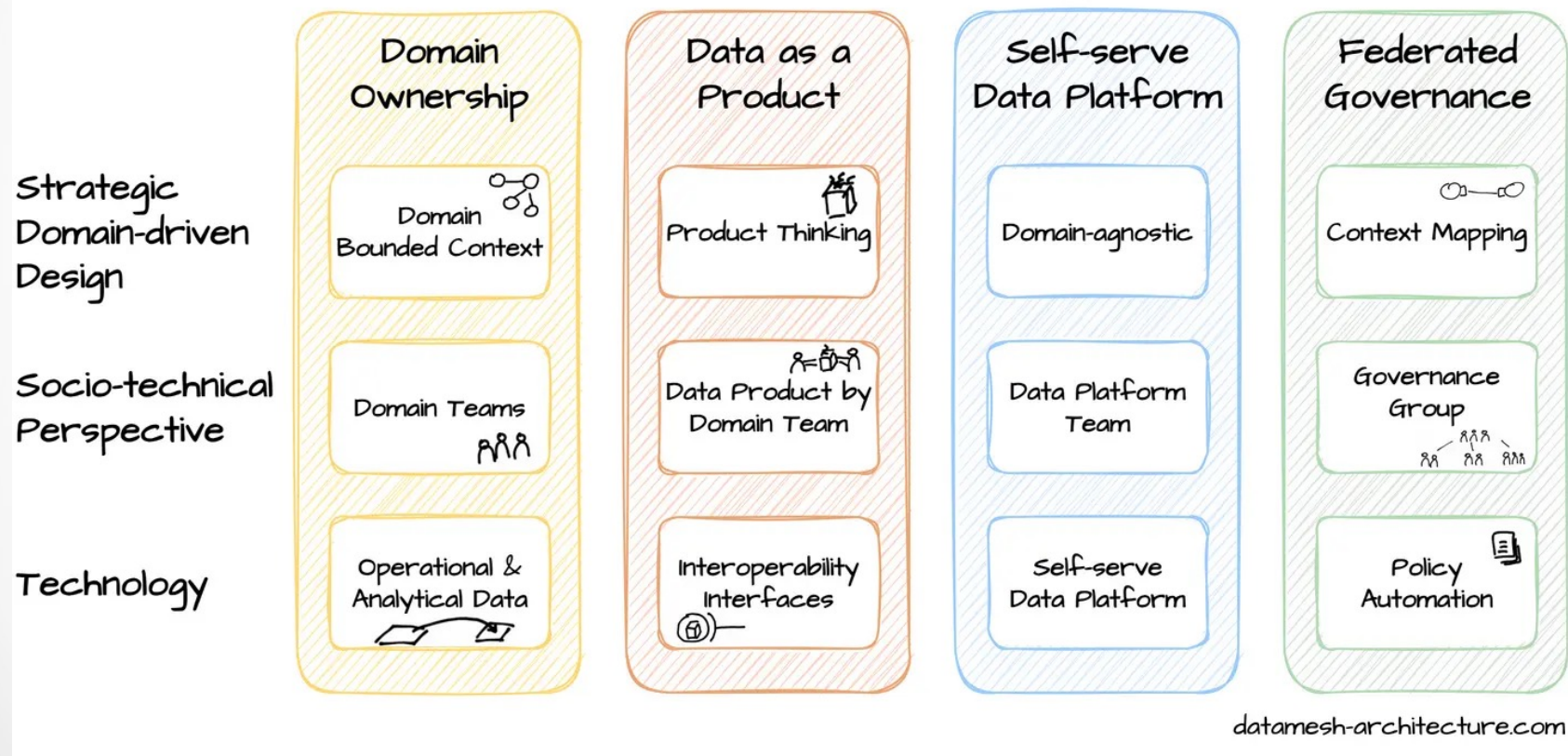
Source: Z. Dehghani, Data Mesh, O'Reilly, 2023

Federated Computational Governance

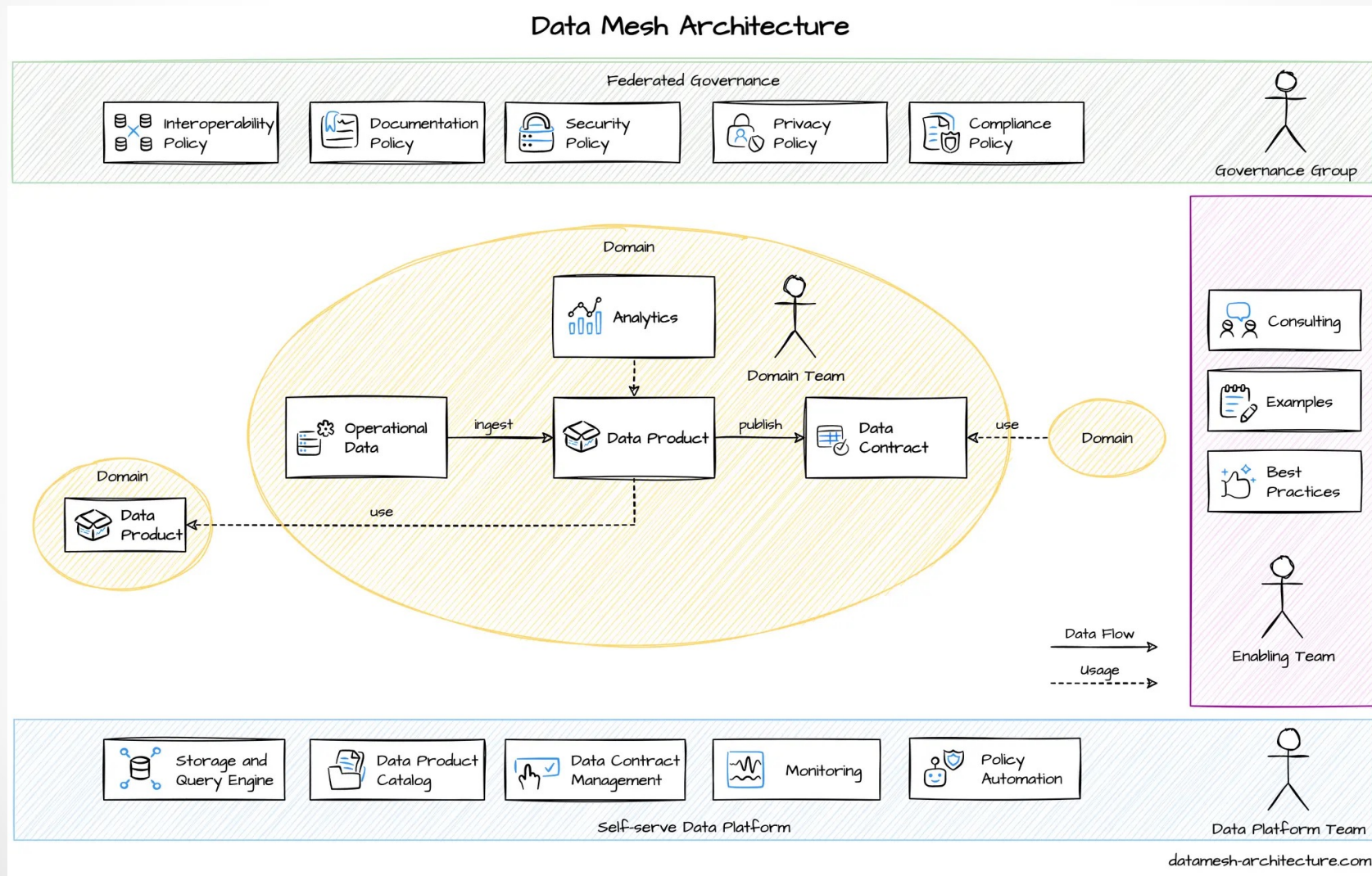


Principles at different levels

What Is Data Mesh?



Data Mesh reference architecture



Another comparison (2)

Data Design Patterns	Data Warehouse	Data Lake	Data Lakehouse	Data Mesh
What it is	A subject-oriented data architecture that integrates detailed data in a consistent way while maintaining a non-volatile history of it.	A set of long-term data containers for managing and refining raw data, using low-cost object storage often delivered from the cloud.	A combination of a data warehouse and a data lake.	A domain-driven data design pattern divided either logically or physically among the teams working in those domains.
Benefits	Generates actionable insights (e.g., in dashboards) from huge amounts of curated data, including the creation of predictive analytics and dashboards that drive operational actions. It aggregates data from all enterprise sources in a central location with consistent governance and supports sandboxes for new idea testing.	Captures previously discarded “dark data” to drive innovation later on and stores data as-is without having to structure it first. The lake also allows insights to be efficiently captured by AI and machine learning services analysing raw information.	Enables an enterprise to systematically extract insights in the mode of a data warehouse — via SQL, machine learning, or any other process — while taking advantage of the vast scale and low costs of a data lake.	Data mesh allows for autonomous active management of data by the teams closest to it and permits increased agility because there’s no central bottleneck. Each team can create its own data products.
Limitations	Not ideal for big data use cases that require the storage and extraction of value from large amounts of raw data, such as that created by IoT devices and web and mobile sources.	Relatively few off-the-shelf tools are available for data lakes, which necessitates significant experience with open-source software. There’s also a high risk of silos due to limited governance, and there can be great difficulty balancing issues between security and ease of access.	Limited agility in adding new features because everything is centralized and monolithic. Data engineers end up spending a lot of their time cleaning up data from teams that have limited incentive to ensure their information is accurate as it goes in.	It’s a relatively new architecture that enterprises are still working out. Performance and governance may suffer because users need to go over the network every time to access different data. Without cross-domain governance and semantic linking of data, it can become very siloed and yield disappointing results.

<https://www.linkedin.com/pulse/data-warehouse-vs-lake-lakehouse-mesh-ashish-bijawat/>

Bibliography

- Principles of Data Integration by A. Doan, A. Halevy, and Z. Ives Morgan Kaufmann
- Xin Luna Dong, Divesh Srivastava: Big Data Integration. Synthesis Lectures on Data Management, Morgan & Claypool Publishers 2015, pp. 1-198
- Clement T. Yu, Weiyi Meng, Principles of Database Query Processing for Advanced Applications , Morgan Kaufmann, 1998, ISBN: 1558604340
- Roberto De Virgilio, Fausto Giunchiglia, Letizia Tanca (Eds.): Semantic Web Information Management – A Model-Based Perspective. Springer 2009, ISBN 978-3-642-04328-4
- M. Lenzerini, Data Integration: A Theoretical Perspective, Proceedings of ACM PODS, pp. 233-246, Editore: ACM, 2002, ISBN: 1-58113-507-6 (This is a scientific paper published in a volume of Conference proceedings)